

Midterm 2 Review: Architecture & Complexity

CMPT 211: OOP and Dynamic Data Structures

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

Midterm 2: Exam Format and Strategy

The Format: Midterm 2 will be a paper-based, closed-book examination. You will not have access to a computer, an IDE, or Python.

The Objective: Execution Tracing

- You must act as the human compiler.
- Do not rely on memorizing syntax! You must be able to read a block of code and manually track how variables, lists, and pointers change line-by-line.
- *Recommendation:* When taking the exam, physically draw out the memory arrays and cross out values as they update.

Review: Object-Oriented Programming (State)

Expect to trace code utilizing our ConcordiaStudent stack. You must understand inheritance (`super()`) and **State Mutation**.

```
def modify_student(student_obj):  
    student_obj.courses["CMPT211"] = 95
```

Core Concept to Review: Objects and dictionaries in Python are passed by *reference*. If you pass an object into a function and change its dictionary, you are altering the original object in memory.

Watch out for questions where multiple variables point to the same object (aliasing), or where external functions mutate a student's grades!

Review: Search Algorithms & Complexity

You must know the theoretical limits of search algorithms and exactly how they behave.

Linear Search:

- Best Case: $O(1)$ (Found at index 0). Worst Case: $O(n)$ (Found at the end, or missing).

Binary Search:

- *Absolute Prerequisite:* The array **MUST** be sorted.
- Worst Case: $O(\log_2 n)$. You must understand why! Every iteration of the loop cuts the remaining search space exactly in half.
- *Tracing Hint:* Know how the boundaries (l**o**b and u**p**b) shift when the target is greater than or less than the midpoint.

Review: Stacks and Queues (CLRS Section 10.1)

We implement Stacks and Queues using flat, contiguous memory arrays.

Stacks (LIFO):

- Tracked with a single `top` pointer.
- Know what happens if you try to pop when `top == -1` (Stack Underflow).

Queues (FIFO):

- Tracked with `head` (where we read) and `tail` (where we write).
- *Tracing Hint:* You must know how to trace the modulo arithmetic that makes the array circular. E.g., `tail = (tail + 1) % length`. Track the integers carefully!

Review: Linked Lists (CLRS Section 10.2)

In Python, we simulated pointers using integer array indices. A Linked List scatters data, linking nodes via explicit Next pointers.

```
# Format: [Value, Next_Index]
memory = [ ["Bob", 0], ["Alice", 2], ["Charlie", -1] ]
```

Core Concepts to Review:

- How to trace the logical sequence of a list by following the integer pointers.
- How to insert a new node. (Which pointer must be updated to maintain the chain?)
- *Complexity*: Why does an Array require $O(n)$ time to delete the first element, while a Linked List requires $O(1)$ time? (Answer: Memory shifting).

Review: Binary Trees (CLRS Section 10.4)

A Binary Tree utilizes dual pointers (Left and Right).

Simulating Trees: We simulated this using lists of lists: [Value, Left_Index, Right_Index]. Be prepared to evaluate nested indices (e.g., `tree[tree[0][1]][0]`).

Traversals & Trees:

- **BST Property:** Left children are smaller, Right children are larger.
- **Traversals:** Know the difference between Pre-Order, In-Order, and Post-Order. (Hint: In-Order traversal of a BST always yields sorted data).
- **Tries:** Understand the structure of prefix trees for string characters.

Review: Heaps (CLRS Section 6.1)

Not all trees require explicit pointers! A Heap is a perfectly balanced tree mapped directly onto a flat array using mathematical indices.

Core Concept to Review: If you have a Heap stored in a 0-indexed Python list, you must know the formulas to find relatives dynamically:

- **Left Child:** $2i + 1$
- **Right Child:** $2i + 2$
- **Parent:** $(i - 1) // 2$

Understand why this provides $O(\log n)$ efficiency without wasting memory on explicit Left/Right integer pointers.

Summary

The exam tests your ability to trace how data structures physically manipulate memory over time.

Review the slides, practice tracing code by hand, and good luck!